

Cadenas y expresiones regulares en JavaScript

String, template literals, Unicode, métodos de búsqueda y transformación, patrones RegExp, grupos, cuantificadores, banderas y validación responsable

SEMANA	SESIONES	DURACIÓN	MODALIDAD
03	05 y 06	4 horas	Presencial

CASO PRÁCTICO

Analizador y normalizador de perfiles

Construirás una aplicación que limpia y transforma nombres, valida correo y teléfono, analiza una biografía, cuenta palabras y puntos de código Unicode, y extrae etiquetas mediante expresiones regulares legibles y acotadas.

Ruta del laboratorio

Avanza en orden. Cada experimento introduce una decisión que luego se aplica en el proyecto integrador.

Bloque	Actividad	Tiempo	Evidencia
A	Preparar el entorno y reconocer String	20 min	Consola y estructura operativas
B	Transformar, buscar y segmentar cadenas	35 min	Ejercicios comentados
C	Comprender Unicode y conteos	25 min	Comparación length / iteración
D	Construir y probar expresiones regulares	45 min	Patrones explicados
E	Crear HTML semántico y accesible	30 min	Formulario funcional
F	Diseñar la interfaz con CSS	25 min	Diseño responsivo
G	Programar la lógica JavaScript	45 min	Analizador completo
H	Probar, depurar y ampliar	15 min	Matriz y reto

Tiempo total estimado: 240 minutos (4 horas).

Ficha del laboratorio

Elemento	Descripción
Logro	Procesa datos textuales y diseña expresiones regulares para búsqueda, extracción, sustitución y validación básica.
Producto	Aplicación Analizador y normalizador de perfiles con HTML, CSS y JavaScript separados.
Estrategia	Experimentos breves, lectura de patrones, construcción incremental, pruebas normales y casos límite.
Requisitos	Visual Studio Code estable y navegador moderno con DevTools.
Conocimientos previos	Variables, funciones, condicionales, arreglos, DOM, eventos y manejo básico de errores.

Resultados de aprendizaje observables

- Distingue una cadena primitiva de un objeto String y reconoce la inmutabilidad.
- Construye mensajes legibles con template literals e interpolación.
- Selecciona métodos de búsqueda, extracción, división y reemplazo según el propósito.
- Explica la diferencia entre unidades UTF-16, puntos de código y caracteres percibidos.
- Interpreta una expresión regular por partes y usa banderas, clases, grupos y cuantificadores.
- Valida y prueba patrones sin asumir que una regex reemplaza la validación de negocio o del servidor.

Seguridad: Nunca insertes texto ingresado por el usuario con innerHTML. En esta guía se utiliza textContent y creación explícita de nodos.

1. Fundamentos de String

Una cadena es una secuencia de unidades de código UTF-16. En la práctica se usa el valor primitivo string; el constructor new String() crea un objeto envoltorio y normalmente debe evitarse.

Código 1. Primitivo frente a objeto

```
const primitiva = "JavaScript";
const objeto = new String("JavaScript");

console.log(typeof primitiva); // "string"
console.log(typeof objeto);    // "object"
console.log(primitiva === objeto); // false
```

Regla: Prefiere literales o String(valor). No uses new String() para representar texto ordinario.

1.1 Literales, escapes y template literals

Forma	Ejemplo	Uso
Comillas simples	'Hola'	Texto simple
Comillas dobles	"Hola"	Texto simple
Backticks	`Hola \${nombre}`	Interpolación y varias líneas
Escape	"Línea 1\nLínea 2"	Representar caracteres especiales
String.raw	String.raw`C:\temp`	Observar escapes sin procesarlos

Código 2. Interpolación y expresión

```
const nombre = "Ana";
const proyectos = 3;
const mensaje = `${nombre} tiene ${proyectos} proyecto${proyectos === 1 ? "" : "s"}.`;
console.log(mensaje);
```

Dentro de \${...} puede colocarse una expresión, pero no una sentencia como if. Si el cálculo es complejo, resuélvelo antes y usa una variable descriptiva.

1.2 Las cadenas son inmutables

Los métodos no modifican la cadena original: devuelven otra. Reasigna el resultado cuando quieras conservar la transformación.

Código 3. Inmutabilidad

```
let curso = " JavaScript Avanzado ";
curso.trim();
console.log(curso);    // conserva los espacios

curso = curso.trim();
console.log(curso);    // "JavaScript Avanzado"
```

2. Métodos esenciales de cadenas

Necesidad	Métodos	Resultado
Limpiar bordes	trim(), trimStart(), trimEnd()	Nueva cadena
Cambiar caso	toUpperCase(), toLowerCase(), variantes Locale	Nueva cadena
Buscar	includes(), startsWith(), endsWith(), indexOf()	Boolean o índice
Extraer	slice(), substring(), at()	Fragmento o unidad UTF-16
Dividir	split(separador)	Arreglo
Reemplazar	replace(), replaceAll()	Nueva cadena
Combinar	template literals, concat()	Nueva cadena
Normalizar Unicode	normalize()	Nueva cadena normalizada

Código 4. Limpieza y segmentación

```
const entrada = " JavaScript, HTML, CSS ";
const tecnologias = entrada
  .trim()
  .split(",")
  .map(elemento => elemento.trim());
console.log(tecnologias); // ["JavaScript", "HTML", "CSS"]
```

Cuidado: split("") trabaja con unidades UTF-16 y puede separar un par sustituto. Para recorrer puntos de código usa for...of o el operador spread.

2.1 slice() frente a substring()

Comportamiento	slice(inicio, fin)	substring(inicio, fin)
Fin excluyente	Sí	Sí
Índices negativos	Cuenta desde el final	Los convierte en 0
Inicio mayor que fin	Devuelve cadena vacía	Intercambia los argumentos
Recomendación	Preferible por semántica consistente	Útil al mantener código existente

Código 5. Extracción

```
const texto = "JavaScript";
console.log(texto.slice(0, 4)); // "Java"
console.log(texto.slice(-6)); // "Script"
console.log(texto.at(-1)); // "t"
```

2.2 Comparación y localización

Las comparaciones directas distinguen mayúsculas. Para ordenar o comparar texto natural considera localeCompare() o Intl.Collator con la configuración regional apropiada.

Código 6. Comparación para español de Perú

```
const comparador = new Intl.Collator("es-PE", {
  sensitivity: "base"
});

console.log(comparador.compare("Álvaro", "alvaro") === 0); // true
```

3. Unicode: qué cuenta length

length cuenta unidades de código UTF-16, no necesariamente símbolos visibles. Un punto de código fuera del plano multilingüe básico ocupa dos unidades; un grafema visible puede contener varios puntos de código.

Nivel	Ejemplo	Cómo aproximarlos
Unidad UTF-16	"👉".length → 2	length, índices, at()
Punto de código	[..."👉"].length → 1	for...of o spread
Grafema	Familia o letra + marca combinante	Intl.Segmenter cuando esté disponible

Código 7. Tres formas de contar

```
const texto = "A👉";
console.log(texto.length); // 3 unidades UTF-16
console.log([...texto].length); // 2 puntos de código

const segmentador = new Intl.Segmenter("es", { granularity: "grapheme" });
console.log([...segmentador.segment(texto)].length); // 2 grafemas
```

Decisión del proyecto: El contador en vivo muestra length porque coincide con maxlength del textarea. El resultado incluye puntos de código para evidenciar que son métricas distintas.

3.1 Normalización Unicode

Dos textos visualmente iguales pueden tener secuencias internas distintas. normalize("NFC") combina representaciones canónicas cuando es posible y facilita comparaciones coherentes.

Código 8. Representaciones equivalentes

```
const compuesta = "é";
const descompuesta = "e\u0301";

console.log(compuesta === descompuesta); // false
console.log(compuesta.normalize("NFC") === descompuesta.normalize("NFC")); // true
```

4. Expresiones regulares

Una expresión regular describe un patrón de texto. Puede escribirse como literal /patrón/banderas o construirse con new RegExp(texto, banderas) cuando el patrón se genera dinámicamente.

Código 9. Dos formas de crear RegExp

```
const literal = /javascript/iu;
const termino = "javascript";
const dinamica = new RegExp(termino, "iu");

console.log(literal.test("JavaScript")); // true
console.log(dinamica.test("JavaScript")); // true
```

Preferencia: Usa literal cuando el patrón es conocido al programar. Si construyes RegExp con texto del usuario, escapa los metacaracteres antes de interpolarlo.

4.1 Anatomía de un patrón

Elemento	Significado	Ejemplo
^ y \$	Inicio y fin de la entrada	/^9\d{8}\$/
.	Cualquier carácter salvo saltos de línea por defecto	/a.c/
\d, \s, \w	Dígito, espacio y carácter de palabra ASCII	/\d+/
[abc]	Uno de los caracteres indicados	/[aeiou]/
[^abc]	Uno que no esté en el conjunto	/[^0-9]/
\p{L}	Cualquier letra Unicode con bandera u	/\p{L}+/u
()	Grupo capturador	/(\d{2})-(\d{2})/
(?:)	Grupo no capturador	/(?:https?):\V\V/
	Alternativa	/rojo azul/
\	Escapa un metacarácter	/\./

4.2 Cuantificadores

Cuantificador	Cantidad	Ejemplo
*	Cero o más	/a*/
+	Una o más	/\d+/
?	Cero o una	/https?/
{n}	Exactamente n	/\d{9}/
{n,}	Al menos n	/\p{L}{2,}/u
{n,m}	Entre n y m	/[A-Z]{2,4}/

Los cuantificadores son codiciosos por defecto: intentan abarcar lo máximo posible. Al añadir ? se vuelven perezosos, por ejemplo .*?. Evita patrones ambiguos con repeticiones anidadas porque pueden degradar el rendimiento.

4.3 Banderas frecuentes

Bandera	Nombre	Efecto
g	global	Busca todas las coincidencias, no solo la primera.
i	ignore case	Ignora diferencias de mayúsculas según reglas Unicode.
m	multiline	^ y \$ también actúan por línea.
s	dotAll	. incluye saltos de línea.
u	Unicode	Interpreta el patrón por puntos de código y habilita escapes Unicode.
v	Unicode sets	Amplía clases Unicode y operaciones entre conjuntos.
y	sticky	Busca exactamente desde lastIndex.
d	indices	Incluye índices de coincidencias en el resultado.

4.4 Métodos que trabajan con RegExp

Método	Devuelve	Uso típico
regex.test(texto)	Boolean	Comprobar si existe una coincidencia
regex.exec(texto)	Arreglo o null	Obtener coincidencia, grupos e índices
texto.match(regex)	Arreglo o null	Recuperar coincidencias
texto.matchAll(regex)	Iterador	Obtener todas las coincidencias con grupos; requiere g
texto.search(regex)	Índice o -1	Localizar la primera coincidencia
texto.replace(regex, reemplazo)	String	Sustituir por texto o función

Método	Devuelve	Uso típico
texto.split(regex)	Arreglo	Segmentar con separadores variables

Código 10. Grupos nombrados y reemplazo

```
const fecha = "2026-08-28";
const patron = /^(?<anio>\d{4})-(?<mes>\d{2})-(?<dia>\d{2})$/u;
const resultado = patron.exec(fecha);

console.log(resultado?.groups); // { anio: "2026", mes: "08", dia: "28" }
console.log(fecha.replace(patron, "$<dia>/$<mes>/<anio>")); // 28/08/2026
```

4.5 Validar no es verificar

Una regex confirma estructura, no existencia ni intención. Un patrón de correo razonable para interfaz no prueba que el dominio o buzón exista. El atributo type=email ayuda, pero el servidor debe volver a validar y aplicar sus reglas.

Dato	La regex puede	La regex no puede
Correo	Comprobar una estructura práctica	Confirmar que el buzón existe
Teléfono	Exigir 9 dígitos y prefijo	Confirmar titularidad o vigencia
Fecha	Comprobar AAAA-MM-DD	Determinar por sí sola si 31/02 existe
Contraseña	Detectar categorías de caracteres	Garantizar resistencia o ausencia de filtraciones

5. Proyecto integrador

La aplicación separará estructura, presentación y comportamiento. No usa bibliotecas externas: el objetivo es comprender las API nativas del lenguaje y del navegador.

Archivo	Responsabilidad
index.html	Estructura semántica, campos, ayudas, salida y atributos de accesibilidad.
css/styles.css	Sistema visual, cuadrícula responsiva, estados de foco, errores y métricas.
js/app.js	Normalización, patrones, validación, extracción, métricas, DOM y eventos.

1. Crea una carpeta llamada lab-semana-03.
2. Dentro crea index.html, la carpeta css con styles.css y la carpeta js con app.js.
3. Abre la carpeta completa en Visual Studio Code mediante Archivo > Abrir carpeta.
4. Activa el guardado automático si lo deseas y mantén DevTools abierto durante las pruebas.

Estructura: lab-semana-03/index.html; lab-semana-03/css/styles.css; lab-semana-03/js/app.js.

6. Implementación de HTML

Abre index.html y escribe los bloques en el orden presentado. Guarda después de cada bloque.

Código 11. index.html — bloque 1 de 4

```
<!doctype html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Analizador de perfiles</title>
  <link rel="stylesheet" href="./css/styles.css">
  <script src="./js/app.js" defer></script>
</head>
<body>
  <header class="encabezado">
    <div class="contenedor">
      <p class="etiqueta">JavaScript Avanzado · Semana 3</p>
      <h1>Analizador y normalizador de perfiles</h1>
      <p class="introduccion">
        Practica cadenas, Unicode y expresiones regulares con datos reales.
      </p>
    </div>
  </header>

  <main class="contenedor cuadrícula">
    <section class="panel" aria-labelledby="titulo-formulario">
      <h2 id="titulo-formulario">Datos del perfil</h2>
```

doctype y lang declaran HTML moderno y español; defer permite que el DOM exista antes de ejecutar JavaScript.

Código 12. index.html — bloque 2 de 4

```
<form id="formPerfil" novalidate>
  <div class="campo">
    <label for="nombre">Nombre completo</label>
    <input id="nombre" name="nombre" type="text"
      autocomplete="name" maxlength="80"
      placeholder="ana maría perez " required>
    <small>Se normalizarán los espacios y las mayúsculas.</small>
  </div>

  <div class="campo">
    <label for="correo">Correo electrónico</label>
    <input id="correo" name="correo" type="email"
      autocomplete="email" maxlength="120"
      placeholder="ana.perez@utp.edu.pe" required>
  </div>

  <div class="campo">
    <label for="telefono">Teléfono peruano</label>
    <input id="telefono" name="telefono" type="tel"
      autocomplete="tel" maxlength="20"
      placeholder="987 654 321" required>
    <small>Acepta 9 dígitos, con espacios o guiones.</small>
  </div>
```

Cada label se asocia mediante for/id. maxlength limita unidades UTF-16 y novalidate permite mostrar mensajes pedagógicos propios.

Código 13. index.html — bloque 3 de 4

```

    <div class="campo">
      <label for="biografia">Biografía breve</label>
      <textarea id="biografia" name="biografia" rows="6"
        maxlength="300"
        placeholder="Desarrolladora web interesada en #JavaScript y #UX."></textarea>
      <div class="contador">
        <span id="contadorCaracteres">0</span>/300 unidades UTF-16
      </div>
    </div>

    <div class="acciones">
      <button type="submit">Analizar perfil</button>
      <button type="reset" class="secundario">Limpiar</button>
    </div>
  </form>
</section>

<section class="panel resultado" aria-labelledby="titulo-resultado">
  <h2 id="titulo-resultado">Resultado</h2>
  <div id="mensajes" class="mensajes" role="alert" hidden></div>

  <article id="tarjetaPerfil" hidden>
    <p class="etiqueta">Vista normalizada</p>
    <h3 id="salidaNombre"></h3>

```

La salida usa `role=alert` para errores y elementos semánticos; `hidden` controla la visibilidad sin eliminar nodos.

Código 14. index.html — bloque 4 de 4

```

    <p id="salidaContacto"></p>
    <p id="salidaBiografia"></p>

    <dl class="metricas">
      <div><dt>Palabras</dt><dd id="totalPalabras">0</dd></div>
      <div><dt>Puntos Unicode</dt><dd id="totalPuntos">0</dd></div>
      <div><dt>Etiquetas</dt><dd id="totalEtiquetas">0</dd></div>
    </dl>

    <div>
      <h4>Etiquetas encontradas</h4>
      <ul id="listaEtiquetas" class="etiquetas"></ul>
    </div>
  </article>
</section>
</main>
</body>
</html>

```

7. Implementación de CSS

Abre `css/styles.css` y escribe los bloques en el orden presentado. Guarda después de cada bloque.

Código 15. css/styles.css — bloque 1 de 5

```

:root {
  color-scheme: light;
  --azul-900: #0b2545;
  --azul-700: #1f4d78;
  --azul-500: #2e74b5;
  --azul-100: #e8f1fb;
  --rojo: #e63b2e;
  --tinta: #172033;
  --gris: #667085;
  --borde: #d7dee8;
  --fondo: #f4f7fb;
  --blanco: #ffffff;
  --error: #9b1c1c;
}

* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: system-ui, -apple-system, "Segoe UI", sans-serif;
  color: var(--tinta);
  background: var(--fondo);
  line-height: 1.5;
}

```

Las variables centralizan el sistema visual y box-sizing hace predecibles los tamaños.

Código 16. css/styles.css — bloque 2 de 5

```

.contenedor {
  width: min(1100px, calc(100% - 2rem));
  margin-inline: auto;
}

.encabezado {
  padding: 2.5rem 0 4.5rem;
  color: var(--blanco);
  background: linear-gradient(135deg, var(--azul-900), var(--azul-500));
}

h1, h2, h3, h4, p { margin-top: 0; }
h1 { max-width: 760px; margin-bottom: .5rem; font-size: clamp(2rem, 5vw, 3.4rem); line-height: 1.05; }
h2 { color: var(--azul-900); }
.introduccion { max-width: 650px; margin-bottom: 0; color: #dbeafe; }
.etiqueta { margin-bottom: .4rem; color: var(--rojo); font-size: .78rem; font-weight: 800; letter-spacing: .08em; text-transform: uppercase; }

.cuadrícula {
  display: grid;
  grid-template-columns: minmax(0, 1.1fr) minmax(300px, .9fr);
  gap: 1.5rem;
  margin-top: -2.5rem;
  padding-bottom: 3rem;
}

```

La cuadrícula crea dos paneles y el desplazamiento negativo relaciona visualmente el contenido con el encabezado.

Código 17. css/styles.css — bloque 3 de 5

```

}

.panel {
  padding: clamp(1.25rem, 3vw, 2rem);
  border: 1px solid var(--borde);
  border-radius: 1rem;
  background: var(--blanco);
  box-shadow: 0 18px 45px rgb(11 37 69 / 10%);
}

.campo { margin-bottom: 1.1rem; }
label { display: block; margin-bottom: .35rem; font-weight: 700; }
input, textarea {
  width: 100%;
  padding: .75rem .85rem;
  border: 1px solid #aeb9c8;
  border-radius: .55rem;
  font: inherit;
  color: inherit;
  background: #fff;
}
textarea { resize: vertical; }
input:focus, textarea:focus { outline: 3px solid rgb(46 116 181 / 22%); border-color: var(--azul-500); }
small, .contador { color: var(--gris); font-size: .84rem; }

```

Los controles conservan foco visible; los botones diferencian acción principal y secundaria.

Código 18. css/styles.css — bloque 4 de 5

```

.contador { margin-top: .25rem; text-align: right; }

.acciones { display: flex; flex-wrap: wrap; gap: .75rem; }
button {
  padding: .75rem 1rem;
  border: 2px solid var(--azul-700);
  border-radius: .55rem;
  color: #fff;
  background: var(--azul-700);
  font: inherit;
  font-weight: 750;
  cursor: pointer;
}
button:hover { background: var(--azul-900); }
button:focus-visible { outline: 3px solid rgb(230 59 46 / 35%); outline-offset: 2px; }
button.secundario { color: var(--azul-700); background: #fff; }

.mensajes {
  margin-bottom: 1rem;
  padding: .85rem 1rem;
  border-left: 4px solid var(--error);
  background: #fdecec;
  color: var(--error);
}

```

Las métricas y etiquetas usan componentes reutilizables; la media query apila el diseño en pantallas estrechas.

Código 19. css/styles.css — bloque 5 de 5

```
.mensajes ul { margin: .4rem 0 0; padding-left: 1.25rem; }
.resultado h3 { margin-bottom: .25rem; color: var(--azul-900); font-size: 1.6rem; }

.metricas {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: .6rem;
  margin: 1.25rem 0;
}
.metricas div { padding: .7rem; border-radius: .6rem; background: var(--azul-100); text-align: center; }
.metricas dt { color: var(--gris); font-size: .78rem; }
.metricas dd { margin: 0; color: var(--azul-900); font-size: 1.45rem; font-weight: 800; }
.etiquetas { display: flex; flex-wrap: wrap; gap: .45rem; padding: 0; list-style: none; }
.etiquetas li { padding: .3rem .6rem; border-radius: 999px; color: var(--azul-900); background: var(--azul-100); font-weight: 700; }

[hidden] { display: none !important; }

@media (max-width: 780px) {
  .cuadrícula { grid-template-columns: 1fr; }
  .metricas { grid-template-columns: 1fr; }
}
```

8. Implementación de JavaScript

Abre `js/app.js` y escribe los bloques en el orden presentado. Guarda después de cada bloque.

Código 20. js/app.js — bloque 1 de 7

```
"use strict";

const formulario = document.querySelector("#formPerfil");
const campoNombre = document.querySelector("#nombre");
const campoCorreo = document.querySelector("#correo");
const campoTelefono = document.querySelector("#telefono");
const campoBiografia = document.querySelector("#biografia");
const contadorCaracteres = document.querySelector("#contadorCaracteres");
const mensajes = document.querySelector("#mensajes");
const tarjetaPerfil = document.querySelector("#tarjetaPerfil");

// \p{L} representa cualquier letra Unicode. La bandera u activa Unicode.
const PATRON_NOMBRE = /^[^p{L}\p{M}]+(?:[ '\-][^p{L}\p{M}]+)*$/u;
const PATRON_CORREO = /^[^s@]+@[^s@]+\.[^s@]{2,}$/u;
const PATRON_TELEFONO = /^[9\d]{8}$/u;
const PATRON_ETIQUETA = /#[^p{L}\p{N} _]+/gu;
const PATRON_PALABRA = /[^p{L}\p{N}]+(?:[''-][^p{L}\p{N}]+)*$/gu;

function colapsarEspacios(texto) {
  return texto.trim().replace(/\s+/gu, " ");
}
```

use strict y las referencias DOM se declaran una sola vez. Los patrones son constantes y se nombran por intención.

Código 21. js/app.js — bloque 2 de 7

```
function capitalizarNombre(texto) {
  return colapsarEspacios(texto)
    .toLocaleLowerCase("es-PE")
    .replace(/(^|[\ ' -])\p{L}/gu, coincidencia =>
      coincidencia.toLocaleUpperCase("es-PE")
    );
}

function limpiarTelefono(texto) {
  return texto.replace(/[\s-]/gu, "");
}

function contarPuntosUnicode(texto) {
  return [...texto].length;
}

function contarPalabras(texto) {
  return texto.match(PATRON_PALABRA)?.length ?? 0;
}

function extraerEtiquetas(texto) {
  const encontradas = texto.match(PATRON_ETIQUETA) ?? [];
  return [...new Set(encontradas.map(etiqueta =>
```

Las funciones puras normalizan texto, teléfono y conteos; ?. y ?? evitan un condicional innecesario cuando match devuelve null.

Código 22. js/app.js — bloque 3 de 7

```
    etiqueta.toLocaleLowerCase("es-PE")
  )]]);
}

function validarPerfil({ nombre, correo, telefono }) {
  const errores = [];

  if (!PATRON_NOMBRE.test(nombre)) {
    errores.push("El nombre solo puede contener letras, espacios, apóstrofes o guiones.");
  }

  if (!PATRON_CORREO.test(correo)) {
    errores.push("El correo no tiene una estructura válida.");
  }

  if (!PATRON_TELEFONO.test(telefono)) {
    errores.push("El teléfono debe comenzar con 9 y contener exactamente 9 dígitos.");
  }

  return errores;
}

function mostrarErrores(errores) {
  mensajes.replaceChildren();
```

La validación acumula errores para que el usuario pueda corregirlos en una sola interacción.

Código 23. js/app.js — bloque 4 de 7

```

const titulo = document.createElement("strong");
titulo.textContent = "Revisa los siguientes datos:";
const lista = document.createElement("ul");

errores.forEach(error => {
  const elemento = document.createElement("li");
  elemento.textContent = error;
  lista.append(elemento);
});

mensajes.append(titulo, lista);
mensajes.hidden = false;
tarjetaPerfil.hidden = true;
}

function mostrarEtiquetas(etiquetas) {
  const lista = document.querySelector("#listaEtiquetas");
  lista.replaceChildren();

  const valores = etiquetas.length > 0 ? etiquetas : ["Sin etiquetas"];
  valores.forEach(etiqueta => {
    const elemento = document.createElement("li");
    elemento.textContent = etiqueta;
  });
}

```

La interfaz se construye con `createElement`, `textContent` y `replaceChildren`; no interpreta el texto del usuario como HTML.

Código 24. js/app.js — bloque 5 de 7

```

    lista.append(elemento);
  });
}

function mostrarPerfil(perfil) {
  document.querySelector("#salidaNombre").textContent = perfil.nombre;
  document.querySelector("#salidaContacto").textContent =
    `${perfil.correo} · ${perfil.telefono}`;
  document.querySelector("#salidaBiografia").textContent =
    perfil.biografia || "Sin biografía registrada.";
  document.querySelector("#totalPalabras").textContent = perfil.palabras;
  document.querySelector("#totalPuntos").textContent = perfil.puntosUnicode;
  document.querySelector("#totalEtiquetas").textContent = perfil.etiquetas.length;
  mostrarEtiquetas(perfil.etiquetas);

  mensajes.hidden = true;
  tarjetaPerfil.hidden = false;
}

function manejarEnvio(evento) {
  evento.preventDefault();
}

```

`manejarEnvio` normaliza primero, valida después y solo entonces calcula métricas y presenta el perfil.

Código 25. js/app.js — bloque 6 de 7

```

const perfil = {
  nombre: capitalizarNombre(campoNombre.value),
  correo: campoCorreo.value.trim().toLocaleLowerCase("es-PE"),
  telefono: limpiarTelefono(campoTelefono.value),
  biografia: colapsarEspacios(campoBiografia.value)
};

const errores = validarPerfil(perfil);
if (errores.length > 0) {
  mostrarErrores(errores);
  return;
}

perfil.palabras = contarPalabras(perfil.biografia);
perfil.puntosUnicode = contarPuntosUnicode(perfil.biografia);
perfil.etiquetas = extraerEtiquetas(perfil.biografia);
mostrarPerfil(perfil);
}

function actualizarContador() {
  contadorCaracteres.textContent = campoBiografia.value.length;
}

```

Los eventos se registran con `addEventListener`. `queueMicrotask` espera a que `reset` haya restaurado los campos.

Código 26. js/app.js — bloque 7 de 7

```

function reiniciarInterfaz() {
  contadorCaracteres.textContent = "0";
  mensajes.hidden = true;
  mensajes.replaceChildren();
  tarjetaPerfil.hidden = true;
}

campoBiografia.addEventListener("input", actualizarContador);
formulario.addEventListener("submit", manejarEnvio);
formulario.addEventListener("reset", () => {
  // reset modifica los controles después de disparar el evento.
  queueMicrotask(reiniciarInterfaz);
});

```

9. Ejecución y verificación

1. Guarda los tres archivos y abre `index.html` en un navegador moderno.
2. Abre DevTools y confirma que Console no contenga errores.
3. Ejecuta cada caso de la matriz y registra el resultado real.
4. Si un resultado falla, coloca un breakpoint en `manejarEnvio()` y observa el perfil normalizado.

Caso	Entrada principal	Resultado esperado
1. Válido	ana maría perez ; ana@utp.edu.pe; 987 654 321	Ana María Pérez; correo minúscula; teléfono 987654321
2. Etiquetas	Biografía: Aprendo #JavaScript, #UX y #javascript	2 etiquetas únicas: #javascript y #ux
3. Nombre con guion	María-José O'Connor	Nombre aceptado y capitalizado
4. Nombre inválido	Ana_123	Mensaje sobre caracteres permitidos
5. Correo inválido	ana@utp	Mensaje de estructura inválida
6. Teléfono inválido	187 654 321	Mensaje: debe empezar con 9
7. Emoji	Biografía: Hola 🇵🇪	length y puntos Unicode pueden diferir
8. Vacío	Biografía vacía	0 palabras, 0 etiquetas y mensaje Sin biografía

9.1 Lectura de los patrones del proyecto

Patrón	Lectura
<code>/^[p{L}p{M}]+(?:['\-\[\p{L}\p{M}]]+)*\$/u</code>	Nombre completo: letras y marcas; después, cero o más segmentos precedidos por espacio, apóstrofe o guion.
<code>/^[^s@]+@[^s@]+\.[^s@]{2,}\$/u</code>	Estructura práctica de correo: contenido, @, dominio, punto y terminación de al menos dos caracteres.
<code>/^9\d{8}\$/u</code>	Teléfono normalizado: empieza por 9 y sigue con exactamente ocho dígitos.
<code>/#[p{L}p{N}_]+/gu</code>	Todas las etiquetas: # seguido de una o más letras, números o guiones bajos Unicode.
<code>/[\p{L}\p{N}]+(?:['\-\[\p{L}\p{N}]]+)*\$/gu</code>	Palabras Unicode; admite componentes unidos por apóstrofe o guion.

9.2 Errores frecuentes

Síntoma	Causa probable	Corrección
Invalid regular expression	Barra o clase sin cerrar	Lee el patrón de izquierda a derecha y revisa escapes.
<code>\p{L}</code> no funciona	Falta la bandera u o v	Añade u al literal.
Solo aparece una etiqueta	Falta la bandera g	Usa <code>/.../gu</code> antes de <code>match()</code> .
<code>test()</code> alterna true/false	Regex global conserva <code>lastIndex</code>	No reutilices g con <code>test()</code> o reinicia <code>lastIndex</code> .
El formulario se recarga	No se llamó <code>preventDefault()</code>	Comprueba manejar <code>Envío(evento)</code> .
La salida muestra HTML	Se usó <code>textContent</code> correctamente	Es el comportamiento seguro; no cambies a <code>innerHTML</code> .

Síntoma	Causa probable	Corrección
Selector devuelve null	id incorrecto o script sin defer	Revisa nombres y defer en script.

Detalle importante: Una expresión regular global o sticky tiene estado: `test()` y `exec()` actualizan `lastIndex`. Los patrones de validación del proyecto no usan `g`.

10. Reto de extensión

Implementa al menos dos mejoras y añade un caso de prueba para cada una.

Nivel	Mejora	Concepto
1	Resaltar en la interfaz si la biografía supera 200 puntos de código.	spread, condición y clases CSS
1	Mostrar las iniciales del nombre normalizado.	split, map y template literals
2	Extraer menciones como @usuario sin confundir el correo.	límites, grupos y matchAll
2	Formatear el teléfono como 987 654 321 con grupos capturados.	replace y grupos
3	Permitir buscar una palabra ingresada por el usuario escapando metacaracteres.	new RegExp y escape seguro
3	Contar grafemas con Intl.Segmenter y compararlos con length.	segmentación Unicode

11. Preguntas de reflexión

1. ¿Por qué `trim()` no modifica la cadena almacenada en una variable?
2. ¿Qué ventaja ofrece un template literal frente a concatenar muchos fragmentos con `+`?
3. ¿Por qué `"a".length` vale 2 y `[..."a"].length` vale 1?
4. ¿Qué diferencia existe entre una clase `[abc]` y un grupo `(abc)`?
5. ¿Cuándo debes usar los anclajes `^` y `$`?
6. ¿Por qué el patrón del correo es deliberadamente práctico y no pretende implementar toda la especificación?
7. ¿Qué riesgo puede aparecer en patrones ambiguos con cuantificadores anidados?
8. ¿Por qué el servidor debe validar nuevamente los datos?

Cierre: Una buena expresión regular no es la más corta ni la más ingeniosa: es la que expresa una regla acotada, puede explicarse y tiene pruebas representativas.

Fuentes de consulta

- [ECMAScript 2026 Language Specification](#)
- [MDN - String](#)
- [MDN - Template literals](#)

- [MDN - Regular expressions guide](#)
- [MDN - RegExp](#)
- [MDN - Unicode character class escape](#)
- [MDN - String.prototype.matchAll\(\)](#)
- [MDN - Intl.Segmenter](#)